Final Report - Racing Game With Physics Simulation and ML Opponents

Karan Dhillon - 3rd Year BSc Software Engineering

Supervisor - Adrian Johnstone

Section 1 - Introduction **4**
Aims/Goals 4
Submission Directory Structure 4
The Problem 5
Survey of Related Lit (Is this literature review??) 5
Introduction 5
Discussion on Machine Learning for this project 6
Project Organisation 7
Software Patterns 7
Validation Methods 7
Vehicle Physics 8
Conclusion 8
Objectives / Milestone Summary 9
Researching 9
Technical Objectives 10
System Design 11
Testing 12
Documentation 12
Rationale 13
Technical Requirements Specification 14
Functional Requirements (Core Deliverables) 14
Non-Functional Requirements 14
Theory Of Features 15
Raycast suspension 15
Tyre Grip 15
Chapter 2 - Tools Used **16**
Tools Used 16
Unity 16
Visual Studio 17
GitLab 18
CSharpier 18
Unity Testing Framework 18
Unity ML Agents 19
Architectural paradigms / design patterns 19
Chapter 3 - Software Engineering **21**
Development Methodology 21
Technical Decisions 22
ML 22
Testing 24
Documentation 25
Chapter 4 - Architecture of the end system **26**
UML Diagram 27
Features of the end system 27

Raycast Suspension 27
Tyre Grip Model 28
Engine and Gearbox Simulation 29
User Interface / Heads Up Display 30
Machine Learning Opponents 31
Modular Vehicle System 31
Vehicle Audio System 32
Control Singleton 33
Cross Data Class 34
SceneBootstrapper 34
Future Work 35
Self Evaluation - ??? 35
Critical Analysis 36
Training time and Environment Complexity 36
Training Set Reward Balancing 36
Deadlines and Time Management 37
What would I have done differently? 37
Chapter 5 - Bibliography 37
Chapter 6 - Project Diary 41
12/10/25 - 19/10/25 41
6/10/25 - 12/10/25 41
29/9/25 - 5/10/25 42
Temporary Hiatus 42
24/11/25 - 30/11/25 42
1/12/25 - 7/12/25 42
8/12/25 - 14/12/25 42
16/02/26 - 22/03/26 42
23/02/26 - 1/03/26 42
2/03/26 - 8/03/26 42
9/03/26 - 15/03/26 43
16/03/26 - 23/03/26 43
30/03/26 - 05/04/26 43

Section 1 - Introduction

This project consists of the development of a 3D racing game, built with Unity. The project implements Proximal Policy Optimisation (PPO) based ML agents as opponents, trained using reinforcement training to provide a significant challenge to the end user, and having the ability to react to the users inputs in real time, as opposed to traditional “rule based” AI opponents.
This project also uses a custom raycast suspension system with a custom tyre grip model, used in place of a traditional Unity WheelCollider setup, allowing for far more granular control over core variables that determine how the vehicles handle. When combined with Unity’s PhysX support, these physics systems will deliver a realistic vehicle physics system.

The expected product of this project is to produce a functional racing game with the aforementioned physics models and machine learning opponents, to work seamlessly in an engaging gameplay loop.

Instructions to run the program are provided in README.md.

Aims/Goals

The aim of the project is to design and develop a racing game, using physics simulation to model the forces acting on a real car, and using Machine Learning agents to be opponents for the end user to race against. By developing custom physics subsystems such as vehicle suspension and tyre grip models to simulate real world forces at play when driving a car on the race track, the project seeks to provide an exhilarating driving experience that feels lifelike while also being approachable for end users of all ages and driving experience level.

Submission Directory Structure

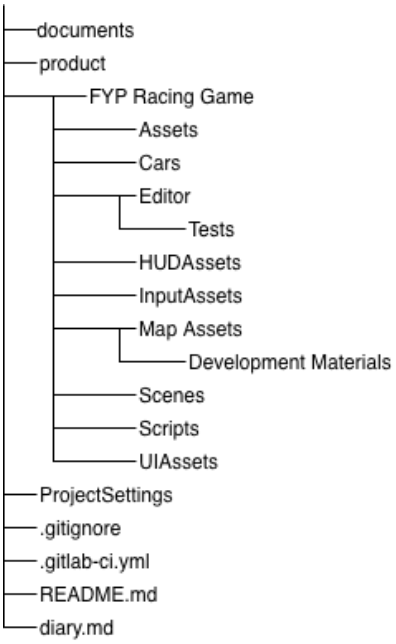


Figure 1: Submission Directory Layout

The Problem

Racing Games have always been dependent on both an engaging gameplay loop, where the player undertakes challenges and races, in order to receive rewards to drive better cars or to unlock other racetracks. In addition, an engaging physics model is needed to make driving the car both enjoyable, while also having the car physically respond as one would in the real world. With advances in 3D graphics, computer physics simulations, and machine learning, racing games have gotten increasingly more advanced, and as such I place a large focus on these areas.

Survey of Related Lit (Is this literature review??)

Introduction

The review examines a variety of methodologies and technologies that can be used in this project. Firstly, a transition from antiquated “CPU” opponents using rule based “AI” such as path tracing, to more advanced machine learning agents that can dynamically react to the players actions in real time. Along with a comparison of machine learning algorithms. Secondly, an investigation into software engineering project management, looking at Agile and SCRUM methodologies. Additionally, some research done into common software engineering techniques and how they related to game development specifically. Finally, a deeper look at the physics and engineering about vehicle dynamics, their relation to basic physical laws such as Newton’s third law, and a justification of a creation of a custom raycast suspension system over unity’s wheelcollider.

Discussion on Machine Learning for this project

During my research I found valuable arguments supporting the use of a machine learning / agent controlled opponent system as opposed to a more traditional pathfinding system used in most racing games. However, due to the possibilities offered by machine learning, such as having an opponent react to the players actions in real time[4] and with the antiquation of traditional “CPU” controlled opponents[5], and the availability of Machine Learning plugins for Unity[1][5][28], there were many paths available to implement challenging opponents to my game.

One of the first choices to make when using Machine Learning is the algorithm to use when training the model. Unity ML-Agents offers two algorithms[1], PPO (Proximal Policy Optimization) and SAC (Soft Actor-Critic). As mentioned in the ML-Agents Theory page, PPO is the default algorithm, due to its suitability for general purpose learning, and improved stability for reinforcement learning[1]. However, SAC is more sample-efficient, requiring up to 50% fewer steps to achieve comparable results compared to PPO[2], which can reduce training times as the model can reach a given experience level with fewer samples. SAC achieves at the cost of higher hyperparameter sensitivity[2][30], which can make initially setting up the model and fine-tuning it far more complex than a PPO based model.

Another variable for fine tuning mentioned in ML-Agents theory is curiosity, which is used to give a signal to the agent where rewards are sparse[1], which can be useful for tracks with complex geometry which could hide reward sources from agents.

Furthermore, using the ML-Agents plugin, I have the ability to Behaviour Cloning, where a certain behaviour is demonstrated to the agent, and used as a base for the agent to learn with. This can provide the benefit of reducing the number of failed initial runs due to the model having some data to “understand” how to drive the car, and how to avoid walls.

However this can be challenging to implement, and on a test run done by Kaunas Technology University, the model was unable to achieve a positive award after 15 million steps.[3] Additionally, by only modelling perfect driving, the model has no data for failure, which can make the agent recovering from a crash or going off track difficult to learn.

In the context of machine learning models, overfitting is a result of an inadequately varied training set, where the model is unable to learn general actions, but rather the “noise” associated with the largely homogenous dataset.[26] As a result of this, model performance will likely collapse when exposed to stimuli outside of its training data set. This can be shown as the model will “memorize all the data, including unavoidable noise on the training set, instead of learning the discipline hidden behind the data.” as Ying [27] notes.

Project Organisation

For a project such as mine, the project management and organisation has many parallels to those being done by larger companies. While many computer science degrees are largely focussed on techniques and technologies used, project management is becoming an increasingly rare skill that many graduates overlook. And as such is now being taught in bachelor level computer science degrees, and business degrees.[6]

I initially gravitated towards Agile Software Development, which is compatible with smaller teams and even solo developers, and has a primary emphasis on software functionality above all else.[11] Furthermore, Agile development aligns with my primary programming paradigm of creating the minimum viable product (MVP) before validating and refactoring a feature, with the ability to respond to changes quickly[7] being critical for solo development where project scope may shrink or expand once a developer finds the limits of their capability and feature set that can be implemented within the set timeframe provided. Additionally Agile Development has the possibility to result in increased productivity due to smaller team sizes[8], although this may not apply to me as a lone developer.

Software Patterns

When developing software projects with a large number of interconnected classes, there are many software engineering patterns and rules to reduce technical debt and increase code maintainability. One of the most important of which is the God Object, where there is a singular class that has multiple responsibilities. A solution to this being the Observer Pattern, frequently cited as a primary solution by being a pattern used to define one-to-many dependencies between classes.[10] This can be used to break down the God object into multiple smaller classes that can abide by the Separation Of Concerns (SOC) or Single Responsibility Principle (SRP).[53]

Furthermore, With the unique software requirements of a videogame, special consideration has to be made for the data flow in a videogame, namely the amount of polling done in a game, usually done for user input, or for gathering data to use in physics simulations. Unity Events can be used to replace polling in most instances[13], however for classes that require constant data updates to refresh, the overhead associated with events make polling unsuitable for these classes.[12]

Validation Methods

The technical foundation of software validation and testing are Unit Tests. Unit Testing involves isolating discrete pieces of logic and testing them against known inputs to ensure the logic runs as intended. The main advantages of unit testing being early defect detection, improved quality, and protection against accidental logic changes when refactoring.[14]

However, Unit Tests are insufficient for complex software projects as the usability of a product is not only bound by the logic that powers it. Many large software companies mandate “integration testing”, which is used to test how well the entire software product works as a whole[15], whereas Unit Testing can only be used to test the logic of a given function or class. In Unity this can be done automatically with play mode tests[16], where the unity game loop is run while running tests to test gameplay features automatically.

Vehicle Physics

Vehicle Suspension has the main priority of maximising the tyre’s contact patch with the ground when driving.[18] Additionally, the Suspension system of a vehicle is to manage the forces imparted on the vehicle through the vehicle’s tyres due to contact with the road surface.[19] As braking, acceleration, and steering apply forces onto the vehicle’s body due to Newton’s third law, the fundamental law of most vehicle dynamics[19], where any forces applied on the road the vehicle is travelling on is also applied onto the vehicles tyres in the opposite direction.[19][20]

Tyre frictional forces are generated as a function of the slip angle, and the angle of the tyres relative to the road.[21] Whereas, to calculate the frictional force an object can impart is (F = mew N), where mew is the frictional co-efficient of the two materials, and N being the normal force being imparted on the object being moved.[22] To model the limits of friction, I could use a traction circle, with which, the total amount of friction Fmax is a function of the coefficient of friction and normal force applied on the tyres. Where, the tyre will never reach the limits of grip unless the lateral and longitudinal friction is less than Fmax (Fmax > sqrt(longitudinalFriction ^ 2 + lateralFriction ^ 2)).[23]

A significant portion of development time for this project will be spent build and refining a car suspension system, and how I can implement the complex motion associated with real life vehicle suspension geometries into my game with advanced algorithms.[17]

Unity has built in systems for vehicle physics, namely the WheelCollider[24], however due to the black box nature of the class, it can be hard to fully understand the logic making the class function.

Conclusion

The research conducted for this project establishes a robust understanding of the theoretical and practical constraints of using machine learning agents trained through reinforcement learning. By evaluating the benefits of using Machine learning as opposed to traditional pathfinding opponent drivers, it was clear to me from the start that implementing machine learning will provide a more dynamic and challenging user experience.[5] Furthermore, the selection of the PPO algorithm over SAC to train my machine learning model, prioritising the stability and aggressive “curiosity” of PPO[2] as opposed to the faster training times of SAC due to higher hyperparameter sensitivity.[2] Furthermore, after reading about the effects of

overfitting highlights the need to vary my training environment to build a more generalised model.[27]

Aside from Machine Learning, investigation into forms of Software Engineering Project management, particularly into Agile and SCRUM[9] helps show the importance of project management in the development and further maintenance of a software product, especially for a lone developer when development time is a limiting factor when determining product scope.

Additionally, research done on software engineering patterns, and validation methods used at larger companies with experience building hundreds if not thousands of successful and reliable software products.

Finally, a dive into the theoretical and functional intricacies of vehicle suspension, building understanding from the basics of Newton's third law[20] and frictional forces, to a model of tyre friction that can be used to build a tyre grip model in my project, and finally, the decision to move away from unity's built in WheelCollider[24] feature due to its lack of extendability, and lack of transparency into its operation. By building my own raycast suspension system, I can implement a more physically accurate simulation, and ensure a high degree of technical transparency.

Objectives / Milestone Summary

Researching

- Modelling of Vehicle Suspension and Tyre Grip: Conduct Research into the forces applied to a vehicle through its suspension, focussing on mathematical models that can be used in this project to replicate the effect of these forces onto the vehicle and its traction.
    - Build up multiple smaller models for spring and damping rates to build a more cohesive yet computationally efficient suspension model
    - Research the relationship between the a tyre's slip angle and the generation of a lateral turning force
- Configuration and underlying hyperparameters of the machine learning: Compare the performance and sample efficiency of Proximal Policy Optimisation (PPO)[29] and Soft Actor-Critic (SAC)[30] algorithms for Reinforcement Learning
    - Research the effectiveness of reinforcement learning techniques such as Curriculum learning[32] and other techniques to minimise compute time when training the agent to navigate complex environments
    - Research hyperparameters for the agent's training, such as curiosity, buffer sizes, entropy etc.

    - Research the factors that create optimal training environments, such as reward and punishment density, and size, and which behaviours should be “taught” to the agent first.
    - Define the objectives that the agent should be rewarded for and punished for, and ensure these rewards and punishments are balanced to ensure the model does not fall into a “local minima” during training.[31]
- Scene Switching and Procedural Level Assembly: Research a methodology to enable scenes to have multiple “configurations” for tracks, where a given map mesh can be used to form multiple track layouts, improving the reusability of map game assets while also providing more content for the end user to play through.
- Modelling of Engine and Transmission system: Research how to break down the complex dynamics of an internal combustion engine and transmission into modular functions. Any models included in this vehicle subsystem should function without excessive computation needed, as this will need to be done every physics tick to ensure accurate vehicle acceleration and engine braking.
    - Model the nonlinear relationship between engine RPM and engine Torque output, and how to form a torque curve in a format that can be easily used in a Unity class. Using a mathematical curve or lookup table can ensure engine torque calculations can be done in each physics tick, while also preserving the “realism” of the vehicle’s behaviour under power.
    - Define the torque multiplication of a vehicle’s transmission and differential, and develop a system to allow unity to apply the torque multiplication of these systems to the engine.[33] This system must be able to dynamically change gear ratios of the vehicle’s transmission during acceleration or deceleration. This will allow the vehicle to react to changes in the player's acceleration input through the transmission down or upshifting.
    - Design an input system for the transmission system to allow the end user to manually switch gears, while also allowing the transmission system to operate without any user input. When operating automatically, the transmission should keep the engine at the highest possible RPM without going over the engine limiter, to mimic the functionality of an automatic transmission in a “kickdown” state.
        - The transmission keeping the engine in the “power band” will be essential for allowing the ML agent controlled vehicle to keep up in pace with a player controlled vehicle with the player manually selecting the gears.

Technical Objectives

- Analysis of Unity Physics Features: Evaluate Unity’s build in physics classes, specifically assessing the WheelCollider[24] class against the advantages and drawbacks of a custom Suspension and tyre model implementation.
    - There must be a focus on the stability and extendability of these implementations when making a comparison of the two, as project scope may expand over time or the project may receive updates, and as such reducing technical debt[34] to a minimum is a high priority.

    - Design and build a system to manage the forces of a vehicle in motion, applying the aforementioned physics onto the vehicle's body, and the computational costs of calculating and applying these forces, as each physics tick must have these forces calculated with high numerical stability for stable vehicle physics. With irregular physics values being sanitised and restricted within reasonable values to prevent glitches and other physics oddities during gameplay.
        - The physics model must be stable as the machine learning agent may pick up "noise" data from the vehicle suspension model's performance, and as such irregular values that may cause the physics system to act abnormally must be filtered out before being applied onto the vehicle.
        - There must be a balance between realism in the vehicles physics and the computation overhead associated with calculating the forces in the vehicle physics model
- Producing a proof of concept for the Vehicle Suspension Model and Tyre Grip Model: Develop a simplistic, working version of the Vehicle Suspension Model and Tyre Grip Model. This prototype will be used to validate the overall "feel" of the vehicle with these subsystems implemented, and to test the realism of the vehicle's physics with spring stiffness, and damping rates from real world vehicles.
- Producing a proof of concept for the Machine Learning Opponents: Train a simple Machine Learning Agent capable of driving a vehicle around a short track. This prototype model will be used as a feasibility study to determine whether the vehicle physics system is compatible with a reinforcement learning machine learning model.
    - A feasibility study is also required to ensure the physics system does not introduce excessive noise to the Agent's model weights when training, as a high Signal-to-Noise Ratio (SNR) can reduce sample efficiency or even deviate model training. As a reduction in sample efficiency can increase model training time to reach a given experience level, sample efficiency must be kept as high as possible to reduce compute time needed to complete this project.
    - Additionally, the basic machine learning agent will be used to experiment with model hyperparameters and reward structures.

System Design

- Produce a list of requirements: Split all project requirements into functional and non-functional requirements, where functional requirements are essential to the project's function, and non-functional requirements are constraints that the project must be confined to.
- Develop a plan with milestones: By implementing small deadlines with each feature's implementation, the project can maintain a structured development timeline that prevents the accumulation of technical debt[34] and ensures that each subsystem is verified and tested before further complexity is added to the project.

Testing

- Implementation of Regressional Unit Testing: Build a series of unit tests using the Unity Testing Framework (UTF) to validate all calculations done in a given class. This ensures that all calculations are done correctly, and remain consistent throughout the development cycle. Ensuring that all functions are deterministic is crucial to ensure vehicle physics are consistent, which is extremely important for reinforcement learning, as the agent will never converge on a given action if there is too much noise from an inconsistent physics system. Furthermore, unit testing will also work as a “last line of defence” to prevent logic bugs being introduced into a refactored class.
- CI/CD Pipeline Integration for Automatic Commit Integrity Testing: Implement a unit testing suite to ensure all code pushed to the repository is tested and confirmed to function as intended. This can ensure that any broken features or logic errors are identified immediately and able to be mitigated early, preventing repository rot.
- Implement Real Time Telemetry and Debugging Features: Utilize built in unity Debugging features such as Debug.DrawRay and other Gizmos, these features can be used to check the functionality of physics related subsystems such as the suspension and tyre grip model during gameplay, which can make validating these systems significantly easier.

Documentation

- Self-Documenting Code: Implement a rigorous documentation standard using verbose class and function names to ensure all code is easy to parse and to make finding the purpose of a function significantly easier. This will make maintaining and extending implemented classes easier by reducing technical debt[34] to a minimum.
    - Ensure all code is documented adequately with code comments, where big functions should be broken down into smaller “chunks” of logic, with each “chunk” having its purpose clearly stated, and its role in the larger class or subsystem also present.
- Technical Journaling: Maintain a developer diary that will document the progress made on the project, rather than just a simple log of tasks. This will provide a detailed history of progress made and allow for critical reflection and adjustment of expectations related to time to develop a given feature.
    - Updating the diary file associated with this project regularly will provide a concrete history of progress for this project. This diary will document the larger changes and evolution of this project, and be updated weekly to ensure that progress is reviewed regularly so action can be taken when unexpected development slowdowns occur.
        - Weekly Diary updates will help with mitigating the risk of development bottlenecks and project scope creep due to the early identification of any issues during development due to the weekly diary update.
        - The Diary will also be used to perform reflective reviews of the progress made on the project during the week, and be used as a tool

to highlight which features may require more development time or are contributing to project scope creep.
    - Using detailed git commit messages will also serve as traceable technical journaling where even small technical changes can be recorded and used as a measure of progress.
- Issue Driven Development: Utilize the GitLab Issue system to break down large features into smaller technical requirements. By using the Issue system, keeping track of each feature's level of development at a glance is far easier due to issue boards.
    - Issues should be short and have a clear expectation of the required actions and expected deliverables, as they should function as small functionality requirements for all subsystems in this project. This will set an objective standard for what determines a function being “completed” and as such will make development much faster as there is no ambiguity about the requirements of a feature’s implementation.
    - Issues should also have an associated branch so any progress recorded by commits can easily be linked to a feature in development through its related issue. This will make measuring progress made on features when multiple features are being developed at once far easier

Rationale

As mentioned previously, with the antiquation of traditional “CPU” controlled opponents[5], and the availability of Machine Learning plugins for Unity[1][5][28], I opted to use Machine Learning for my opponents.

When choosing a machine learning algorithm, I had a big emphasis on using a general purpose algorithm, as choosing an algorithm with specialisation in a specific task can restrict me in terms of features to be implemented. Furthermore, due to the chaotic nature of the agent having to interact with the car while it is also being interacted with by the physics system, a stable algorithm was also desired. Furthermore, while training time and compute power required was a concern (an application where SAC would excel)[2], the sensitivity to small changes in hyperparameters would make initial model setup significantly more complicated and waste limited development time. Additionally, due to the complexity of the environment, a Machine Learning algorithm that is more aggressive with discovery would also be more suitable, as PPO exhibits this more than SAC[2], This further made PPO the most suitable algorithm.

Additionally, I decided to not use Behaviour Cloning and instead have my Machine Learning Agent learn through reward based reinforcement learning, due to the challenges involved with getting an agent to learn through Behaviour Cloning, as shown by Kaunas Technology University[3].

While researching Agile methodologies, SCRUM[9] is frequently cited as an effective methodology to manage software engineering projects without certainty[8]. This made SCRUM much more appealing to me due to the number of new technologies I plan to

implement in this project. This is due to the repeated expectation of reviewing progress made[9] which can help make problems during development easily apparent.

Given the lack of transparency related to the WheelCollider[24] and its functions, I decided it would be better to implement my own raycast suspension system using the unity raycast system.[25] Furthermore, building a custom suspension system also allows for far more extendibility compared to a the unity provided solution, such as being able to add custom tyre friction curves, or adaptive dampers. Furthermore, being able to extend the data from a custom suspension class will be useful for making other suspension related classes work more accurately as information can be pulled from the class directly with getter methods.

Technical Requirements Specification

Functional Requirements (Core Deliverables)

- The Product must contain a suspension and tyre grip model to provide a basis to construct a car controller
- The Suspension model must not “clip” through the ground to avoid unfair play or glitches for the machine learning opponents, as the Machine Learning agents are sensitive to noise in training data, and suspension glitches will make training significantly harder
- Machine Learning Opponents must be able to traverse the track without driving off the track or colliding with other drivers. This is mandatory to provide a fair experience to end users.
- There must be a visual debugging system to monitor physics subsystems while running the gameplay loop for validation testing.
- There must be a UI to enter the game, select a track and transport the player to the track in their vehicle. This is important to bolster usability of the software, as most end users will be used to clear, simple videogame UI design
- There must be a HUD for the user to view vehicle information, lap times, and position in a race. This is essential for the user to accurately monitor their speed for braking zones and steering, and engine RPM must be visible as the user will need it to time their gear changes

Non-Functional Requirements

- All physics calculations must have a small computational footprint to be able to be run within each physics tick to ensure the game is playable
- The Vehicle physics system must be deterministic in nature in order to have a suitable foundation for the agents to learn how to operate a vehicle competitively

- The UI must be easy to use and be clear in its operation without external instruction needed for an average end user to operate the menus in the game

Theory Of Features

Raycast suspension

The suspension on most conventional road vehicles consists largely of a spring and damper system. Without a suspension system it would be incredibly challenging for a vehicle to ensure the tyres have consistent contact with the ground, and make vehicles incredibly uncomfortable to operate and travel in.

Of the two main suspension subsystems, the spring is used to absorb the kinetic energy associated with the wheel being pushed up or dropping down into any imperfections in the road's surface, and to store that energy into Elastic Potential Energy in the spring.[35] The spring is also used to force the wheel into the ground using the weight of the vehicle.

Whereas, the damper is used to remove energy from the spring when oscillating. This is used to ensure the spring does not extend and compress virtually infinitely when loaded with energy, this can reduce the amount of force applied on the tyres if left uncontrolled, so dampers are added to slowly remove energy from the system and stabilise the vehicle.[36]

However, energy should not be removed from the system immediately, as this prevents the spring from being able to articulate the vehicle's suspension to the road surface the vehicle is travelling on, this is known as "Overdamping" and negatively affects both vehicle traction and ride comfort.

The Force applied by a spring can be calculated by using Hooke's Law ($F = k\Delta x$), with a known spring constant (k) and distance (Δx) the spring has stretched or compressed compared to its rest length, The calculated force is the force that will be applied on the vehicle's chassis.

In most vehicle suspension systems, damping is more aggressive when the spring is oscillating quickly. This is because most dampers are hydraulic, where the resistive force is generated from forcing incompressible hydraulic oil through constrictions in the piston of the damper,[37] with the piston travelling through the hydraulic fluid with a higher velocity, the more resistance the damper provides. This is called viscous damping, which can be modelled by using the damping rate of the damper, and the speed of the change in spring length ($\Delta x/t$).

Tyre Grip

However, the frictional limits (and therefore the limits of the tyre's grip) is not independent between its two directions, using the grip of the tyres to generate a lateral turning force means there is less grip available for slowing down (braking) or accelerating the vehicle. This is modelled with the friction circle, which shows the maximum magnitude of grip in both directions given a unit vector and magnitude. This amount of grip can be modelled using Kamm's Circle, where the total force applied on the wheels ($\sqrt{Fx^2 + Fy^2}$) is less than the total grip available to the tyre ($Fmax$).[38]

Tyres have two main dimensions of grip, that being the longitudinal friction of the tyres and the shearing / sliding resistance. The frictional force the tyres can provide (and by extension, the grip available to the vehicle) is a function of the normal force applied to the tyre, and the coefficient of friction.[39] This can be demonstrated by having a vehicle apply the brakes, as the vehicle decelerates, the weight of the vehicle shifts to the front wheels, this increase in the normal force applied on the tyres, provides a larger frictional force (and by extension grip) and allows the vehicle to turn more aggressively.

To steer a vehicle, the tyres need to provide a lateral frictional force to force the front of the vehicle to turn in the direction of the tyres, this is provided by the different coefficient of friction provided by the tyres, where the axis on which the tyre rotates will have a far lower friction than the axis on which the tyre will shear against the road surface. This difference in frictional force across the two directions will impart a force on the front of the car, pushing it into the turn. The difference in angle between the direction of vehicle travel and the angle of the tires is called the "slip angle", which is what determines how much lateral force is generated[40] as the magnitude of the shear force is determined by how large the slip angle is.

Chapter 2 - Tools Used

Tools Used

Unity

There are many game engines free for personal / academic use, and I had narrowed down my initial choices to Unreal[41], Unity and Godot[42]. I initially wanted to use Godot as it was extremely lightweight and an open source alternative to Unity and Unreal. However, due to limited support due to the open source nature of the project, and lack of online tutorials on how to use the engine, I decided to not use Godot. Unreal engine has the advantage of being capable of breathtaking visuals, and has a visual scripting to dull the extremely sharp learning curve associated with an advanced game engine, however, due to how challenging it is to work with a visual scripting language for more advanced classes, and the challenges of using C++ as opposed to the far simpler C# programming language. As such, for the development of my project, I opted to use Unity as my game engine of choice.

One component of my decision to use Unity is the extensive use of C# throughout the development process. As C# is an object oriented programming (OOP) language, Encapsulation of logic is extremely simple and can help reduce the odds of scripts interacting with data they shouldn't be interacting with.[43] This has multiple advantages, one of which being Decoupling, as each class can be Programmed to work independently of one another which allows changes in a given class to not affect any others, making tracking down bugs and unit testing my code significantly easier.[44] Another advantage of Encapsulating Logic is in a large scale project such as a video game with a large array of functions, Encapsulating Logic and strictly using access modifiers can help cut down on this.

Additionally, Unity uses the Nvidia PhysX framework for advanced physics simulations.[46] This both makes the physics much more realistic while eliminating any time needed to make my own physics system as the Unity physics package is already built into the engine, and there is plenty of documentation and tutorials available on how to use the package.

Furthermore, C# is a highly flexible programming language which can be compiled to Common Intermediate Language (CIL),[47] this can then quickly be compiled to native machine code using Unity's Just-In-Time Compiler, or done ahead of time using Intermediate Language 2 C++ (IL2CPP) for the final build.[48] This reduces CPU overhead massively and can allow my project to grow in complexity while also keeping computational efficiency acceptable.

Another Component is how highly Component based Unity is, with GameObjects being composed of meshes and scripts, this flexible architecture allows for far more control over what features a given object or vehicle in my project has, with classes being able to be added to base game objects. This modularity allows me to use composition to build objects from small reusable classes and have the ability to assemble them dynamically for different use cases in my game.

Visual Studio

For an IDE, I opted to use Visual Studio Code as there is a large library of extensions, such as native Unity[49] and C# Extensions for code highlighting and utilising Unity specific analysers and refactorings. Other Extensions used during this project include the CSharpier Extension to allow me to have my code automatically formatted into the CSharpier format. One powerful feature built into Visual Studio Code is Intellisense,[50] which is an advanced code completion tool that can "understand" the context of the Unity API (A feature provided by the Unity VS Code Extension). This feature can warn the user of inefficient code before switching back to the Unity Editor, which can deter developers from writing code that can cause performance issues.

Furthermore, Visual Studio Code is free to use and also has native Git Integration built in, this allowed me to quickly manage my code, see where I am working across my branches in an extremely user friendly format, which further saved development time.

GitLab

GitLab was the designated Version Control System (VCS) for this project, this gave me access to far more integrated tools compared to using Git through GitHub.

Using Issues to track my progress and to keep me focussed on what was required to make the next step in my project helped my sense of progress during my project.

Furthermore, access to using Continuous Integration / Continuous Deployment (CI/CD)[45] pipelines also helps ensure my code is up to standards set and tested at every commit to my repository, this helps me catch bugs or code that is not compliant with a set coding standard.

The main advantage of using GitLab (and Git as a whole) is the ability to have snapshots of my work saved to my machine and an offsite server (RHUL GitLab Server), this allows me to go back to a known working version of my code if I break a feature and am unable to fix the issue. This is invaluable for cutting down wasted development hours on trying to go back to a known working state of my project, on the off chance a feature I intend to implement has unexpected dependencies or requires a completely different approach.

Another advantage of using GitLab through the RHUL GitLab server is the data protection offered by going through my University as opposed to a normal github account. This ensures my intellectual property is stored securely in an environment managed by my academic institution instead of a third party commercial server.

CSharpier

For my CI/CD Pipeline, I opted to use CSharpier to regulate the style of my committed code. Using Code Style enforcement ensures that my code is formatted in a legible and easy to read, avoiding legibility issues such as excessively long line lengths. Implementing this into my CI/CD Pipeline also prevents my repository from being filled with poorly formatted code as a Pipeline failure is alerted to the user through GitLab. These issues are also alerted to the user when attempting to make a merge request, further reducing the chances of poorly formatted code being pushed to my repository.

Furthermore, using a deterministic code style enforcement ensures that any clashes in git branches are not due to formatting issues but instead due to actual changes in logic or comments in my code. Additionally, moving all formatting responsibility to CSharpier reduces cognitive load related to code formatting as it is all done automatically and to the same standard, making CSharpier an invaluable tool for cutting down development time on code formatting.

Unity Testing Framework

Another advantage of using Unity is the ability to use the Unity Testing Framework (UTF)[51] to unit test my code, with the ability to test gameplay related code by having tests conducted within a running game instance.

Being able to quickly write tests with many cases and have these tests be able to be run directly in the Unity Editor makes testing code extremely easy and fast, which further saves time when developing and allows me to run tests more often during development.

Compared to NUnit,[52] which UTF is based on, UTF has the ability to test MonoBehaviour Scripts. Whereas, NUnit can only test C# scripts that are not dependent on any Unity Features at all, which pushed me away from NUnit for my unit testing needs.

However, UTF requires a license of unity and a running unity instance; This makes implementing a Unit Testing check on my CI/CD pipeline extremely difficult without a second unity “seat”, and with the resources needed to run the Unity Instance, this was a major drawback of deciding to use UTF, and as such all Unit Tests were run locally before commits were made. Whereas, NUnit is far lighter and easily able to be run on a CI/CD pipeline, which would extend the protection of unit testing to every commit sent to my repository.[45]

Unity ML Agents

I opted to use Unity ML Agents as the plugin is built directly for Unity and extremely easy to implement to my project,[1] which allowed me to focus on the training of my model for my project as opposed to having to work on an implementation for ML agent training. Furthermore, as ML Agents trains the model within the game itself, other game subsystems such as vehicle physics are used, and the data the model intakes from the game is affected by these subsystems which can make the end model far more useful in game as the model is not trained in a mathematically perfect “vacuum” disconnected from other game subsystems.

Furthermore, the ML agents plugin has the option of using Heuristic modes[1] that allow developers to use player inputs to check for reward balancing by taking the place of the AI agent. This makes reward balancing significantly easier as testing can be quickly done by a human developer instead of wasting compute time on ML training runs that are doomed to fail and not produce a usable model.

Architectural paradigms / design patterns

As mentioned previously, my project heavily relies on a component based architecture, without a set hierarchy, using multiple modular classes to build usable game assets. This allowed me to build “skeletons” with only the base information required for the vehicle to function, such as vehicle data, suspension raycast positions, etc. and to have other modules such as User Input, ML Agent Raycasts, Cameras etc added when the vehicle is spawned into the game.

This heavily reduces the amount of unnecessary code being run as only modules related to the vehicles function is added to the vehicle at spawn time, and also improves the modularity of my car controller as multiple vehicle “skeletons” with differing meshes and data can be made, and all other functions can be added at runtime, which also cuts down on code duplication across these game assets.

Furthermore, this component based architecture helps to enforce a Separation of concerns (SoC) and pushes me to focus on classes through the Single Responsibility Principle (SRP),[53] where I broke down “God Classes” into smaller modules that are both easier to code and far easier to test, as all functions are separated and a bug introduced into one class does not affect any of the other classes, or “modules”.[54] For example, breaking down my Car Controller Class to separate User / Agent Input, Power Calculation, and Force Application into multiple classes allows for far easier testing and can increase code maintainability as classes are not overwhelmingly long and complex.

Additionally, I implemented a service class, called CarCalculations. I used this class to congregate all vehicle related calculations into one class. This SSOT for calculations cuts down on any potential duplicate code, and makes code maintainability far easier on the classes where the calculations are required, as using a detailed method name can communicate the calculations done, and if any information about that function is needed, it is all in one class.

Another advantage of implementing a service class is that as this class is virtually separated from unity, the class was not a MonoBehaviour class, which means this class can be unit tested by NUnit, without a running Unity instance required.

For Interactions between classes, I initially used Explicit Associations, this allows me to select each class directly which made development much faster as these connections were permanent and extremely easy to implement. Furthermore, using these Static Assignments to link classes provided Compile-Time Safety as Intellisense could catch misspelt function or class names. This further reduced development and made initially testing the function’s functionality extremely easy.

However, this approach can create significant technical debt[34] when applied to large projects, such as mine. Therefore, once I had verified that the newly created feature or class was functional, I moved over to using events as opposed to Explicit Associations.

One disadvantage of Explicit Association which pushed me towards moving to using an Event-Driven Architecture, or an Observer Pattern. This improved the scalability of my existing classes as a virtually unlimited number of listeners can be subscribed to an event.

Another Disadvantage is the increased coupling between classes, as they are required to have a reference to a different class to function, this can increase fragility in my code, as everything can have dependencies that are not immediately obvious when looking at the code, so if a function or variable is renamed, it can cause a “ripple” effect throughout my code. Whereas, using events side steps this issue entirely.

However, using an Observer Pattern through unity events is unsuitable for communication that is repetitive, otherwise known as polling. For physics related calculations, there is too much overhead when working with an event as opposed to calling the method associated with a different class directly. This is because invoking an event as opposed to calling a method requires more work from the engine, including nullchecks and iterating through a list of event listeners, whereas directly calling a method side steps all of this entirely.

I believe that the improvement in performance for polling operations is far more beneficial than the increase in coupling between the UI, Physics and User Input related classes.

Chapter 3 - Software Engineering

Development Methodology

When developing a given feature for my project, I opted to use a rapid prototyping approach to build a minimum viable product (MVP) before refining the logic of my code and refactoring and documenting.

I broke down my workflow into three distinct steps, a “Prototyping Phase” where failure was expected to be quick and common, where ideas to solve a problem could be hacked together and tested to find any flaws in a sandbox scene built for testing.

The next phase was a “Verification Phase” where deeper testing was done to ensure the feature I had written was given more scrutiny and aggressive testing to uncover any edge cases that my code would react poorly to. This was also done with Boundary Unit Testing for functions that are not inherently gameplay related, but more calculation focussed.

Thirdly, a “Refactor and Document Phase”, where all code was reviewed and compacted down to densify code, furthermore, all methods were broken up into smaller chunks where possible with each chunk corresponding to a function done. Additionally, any methods with excessive responsibilities were then broken up into separate classes to abide by the Single Responsibility Principle (SRP), as writing unit tests will be significantly easier with classes that have a sole responsibility and minimal dependencies on other classes.

The biggest advantage to this approach is speeding up development time as secondary requirements such as documentation were done once the functionality was verified to be in line with the required actions / expected deliverables outlined in the feature’s relevant issue on GitLab.

Another advantage was time saved from having to refactor or document code that may have been moved or deleted in favor of better code, as the time spent doing so would have essentially been wasted.

Secondly, by ensuring that code was refactored and documented once functionality was verified, I was able to keep technical debt[34] incurred by a rapid prototyping development process to a minimum.

Technical Decisions

ML

When initial prototype models were trained, my main focus was to maximise the speed at which the agent could traverse the track. Therefore, I placed strict penalties and forced the agent to end the episode when the vehicle collides with the walls of the track.

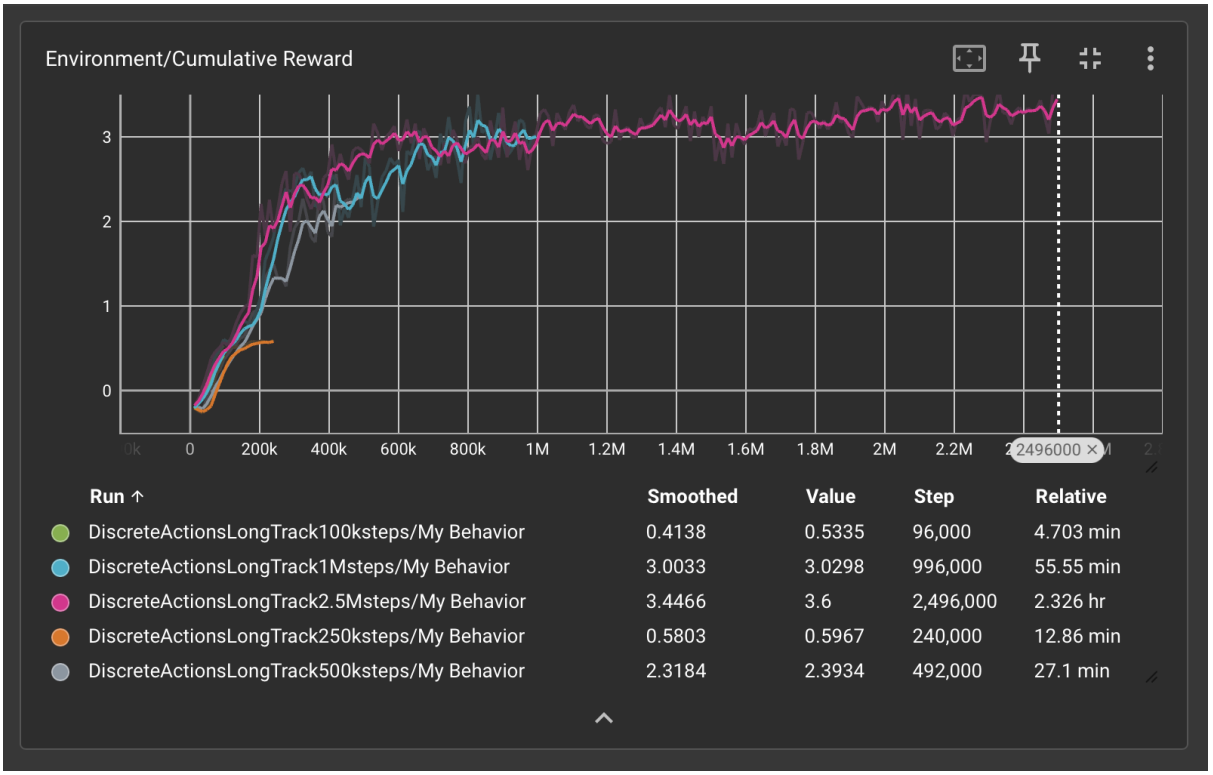


Figure 2: Multiple training runs on a “long track”, showing a plateau in rewards around 700,000 steps

Due to the relative simplicity of the model, training times were minimal, with reward stabilisation becoming evident after around 750-800,000 steps in training.

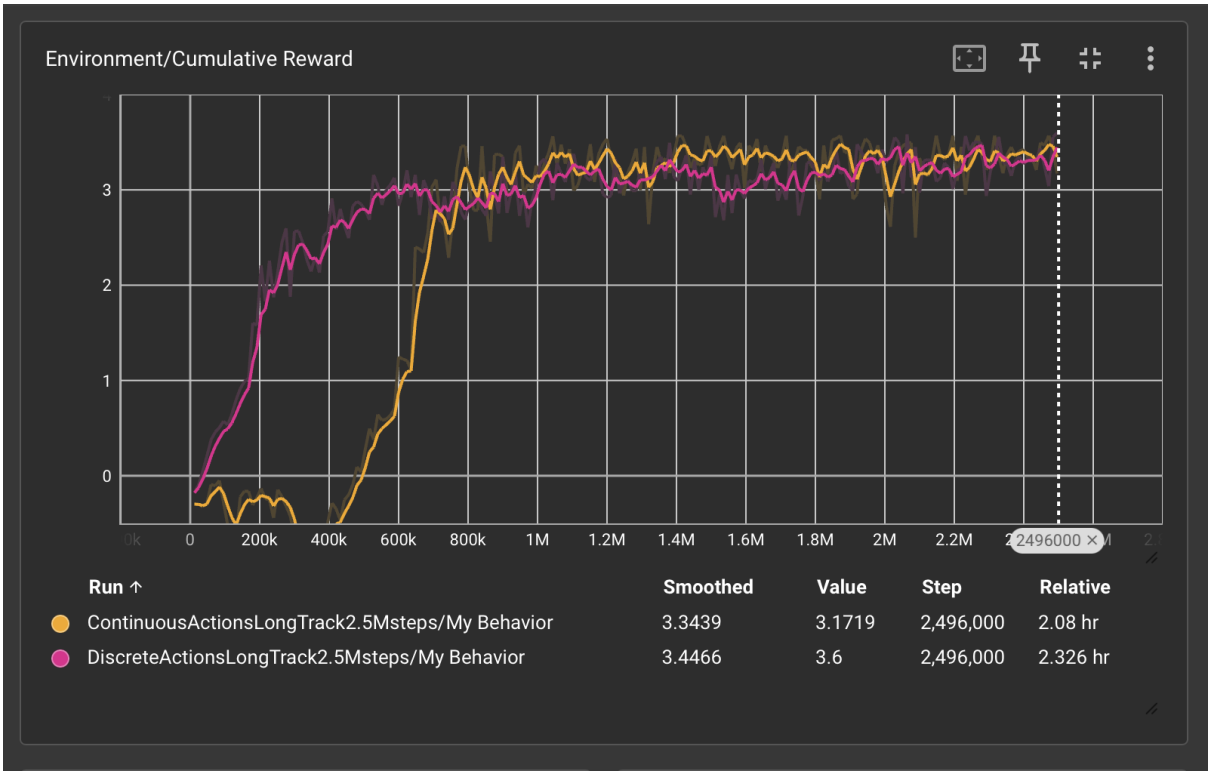


Figure 3: A Comparison between the steps required to produce an “experienced” model with a discrete and continuous action space

One decision I had to make early in my prototyping and testing stage was to use a continuous or discrete action space. As with a continuous action space the model has the ability to have more fine control over the steering, braking, and throttle input, putting the model on par with a user playing the game with a gamepad in terms of input granularity. However, due to the extreme action space size associated with this, model training times were significantly longer and any increase in difficulty for the AI training would accordingly take exponentially more time to train a model to overcome these challenges with a continuous action space as opposed to a discrete one.

Furthermore, to further reduce the action space and speed up model training, I collapsed down the throttle and brake inputs from two discrete branches of size 2, and instead into a single discrete branch of size 3, coupled with only using ML agents in conjunction with my car controller with the “simple automatic” transmission option. This further speeds up training without any noticeable degradation in model quality to the end user, while also giving the model the option to reverse, instead of just brake.

Therefore, early in my ML development journey I opted to exclusively build models with a discrete action space.

Additionally, utilising layer masks allowed me to separate my raycast perception sensors, to have checkpoint detection work independently of wall / obstacle detection, which allows me to “filter” out noise objects, further improving model training speed as scenes could be “simplified” to be either a wall, vehicle, or a checkpoint.

Later in my ML development, I was still bound by the observation space of the model, as having too many observations fed into the model led to a large variation of data fed into the model, which can make it challenging for the model to identify patterns in the input data to react to. Therefore I took further action to reduce the observation space, by simplifying my raycast perception system from 3 sensors to one, by combining vehicle and wall detection and instead feeding the difference in position and direction to the next checkpoint to the model directly, I could improve model training speed, and improve the precision of the obstacle raycast detection by having the capacity to add more raycasts in the remaining sensor.

Balancing the complexity of the model's perceived environment and the compute power required to train the model was a continuous challenge throughout the development of my game.

As mentioned in my literature review, Model overfitting is a challenge when working with reinforcement learning models, where the model will "memorise" the noise within its training environment and have performance collapse when used outside of that environment. To help vary the environments my model was trained in, I implemented randomised obstacles, spawn locations and rotations. This ensured that my model was put in a variety of starting conditions and environments, which will help in managing model overfitting.

Additionally, to manage "cold starts" when training a model in a complex environment, I opted to use Curriculum Learning. By training the model to complete simple tasks in a more simple environment and slowly increasing difficulty, I could reduce training time associated with having the model understand how to operate the vehicle.

One challenge I experienced when training my models was the issue of the model finding an unexpected local minima when training, where the model would refuse to take any actions as any actions would result in negative rewards, and the model would not complete an action that gave a small negative reward to then complete an objective that granted a positive reward, such as passing a checkpoint. To combat this, I ensured that my Model was exposed to a mixture of positive and negative rewards, and slowly fine tuning reward values to incentivise / disincentive given behaviours.

Additionally, one method that could be used to reduce model training times was parallel training, where a multitude of identical environments are run in parallel to reduce training time. This can help take better advantage of hardware such GPUs. Therefore, I opted to use parallel training to speed up model training and allow for more rapid prototyping of reward structures.

Testing

My Testing Strategy for this project largely centred on playtesting individual features using specialised "development" scenes in my unity project, called sandbox testing. By building simple scenes to playtest features as opposed to using play mode tests using Unity Testing

Framework (UTF), testing was far easier and faster, which allowed me to move faster when developing features for my game. Furthermore, playmode tests are extremely complex and time consuming to set up in a way that can create a meaningful line of protection from bugs, which led me to adopt playtesting as my primary method to test gameplay related features.

Initially, I wanted to use NUnit for Assertion-Based testing, where predetermined inputs are run through a function to produce a known result. One advantage of NUnit was the ability to implement it into my CI/CD pipeline through GitLab, which could alert me if I had inadvertently broken a feature in my codebase without realising.

As NUnit cannot interact with MonoBehaviour classes, I opted to move all calculations to a separate service class, called CarCalculations. This not only concentrated all calculations into an easy to test and modify class, it also reduced coupling[44] from classes having to "piggyback" off other classes.

Additionally, implementing NUnit into my CI/CD pipeline caused a multitude of issues, mostly related to conflicting configuration files. Therefore I opted to use UTF unit testing and to do all testing locally instead of trying to make the CI/CD pipeline to work with custom configuration files.

For my testing strategy, I used three distinct groups of tests to ensure a wide number of scenarios can be accounted for. One strategy was to use the exact same logic as the function I am testing, this "logic test" has the role of ensuring the function works as it worked before, which offers protection in the case of refactoring, where finding bugs introduced into the function through refactoring can be challenging as often code logic is condensed down massively.

Another testing strategy was to use erroneous values to test the implementation of clamps and limits in my code. By using these erroneous values in testing, I could test for functionality in cases that I may not have thought of, which usually caused my functions to fail. This forced me to consider error handling in my functions which improved the stability of my codebase massively.

Finally, I used specific value testing, where I used a small selection of input values, and asserted the result with hand calculated values. This was used to see how my functions would operate regarding floating point precision issues.

Documentation

In order to improve the maintainability of my code, I opted to use code comments on each subsection of each class. By breaking my code up into "chunks" of self-contained code that can be explained with a short comment, it allows me to modify my classes with ease as each chunk of code can be identified by its function and as such reduces technical debt[34] massively.

Furthermore, for larger subsections of code that could not be broken up into distinct chunks, comments were added on each line explaining how the function works, what each line of code is doing in plain english to aid understandability from a glance, and explaining the dataflow of any functions. The biggest advantage to doing this is to accurately convey any insights about the code related to the larger subsystem it is used in. This makes chasing bugs faster as specific calculations are labelled and any processed data is labelled with what class processed it and what was done.

Additionally, any information related to the configuration of the class or any dependencies were added at the top of each class, clearly visible for me or any future developers to set up these classes for testing.

Furthermore, I documented my progress and changes in my codebase using Git Commit messages. This allowed me to look back through my commits if I needed to go back to a specific stage of development, and allowed me to compare my code to a previous state if a previously fixed error crops back up.

For the File Structure of my Unity Project, I Opted to separate all my assets by function, and having all scripts stored in a separate “Scripts” folder. This allowed me to easily find assets related to a specific function, such as the HUD, Input, or Car assets without having to search through large and generically sorted folders.

Additionally, I kept all scripts organised by which subsystem of my project uses them, with scripts related to cars, UI, Scenes etc all being stored in separate folders.

An advantage of this approach is that all prefabs used in my project were all stored in a singular easy to find place, which made asset reference errors rare and easy to fix, and ensured a single source of truth (SSOT)[55] for every gameObject, which prevents duplicate assets being stored in my project, cutting down on wasted storage space and confusion when managing assets in my game scenes.

Chapter 4 - Architecture of the end system

UML Diagram

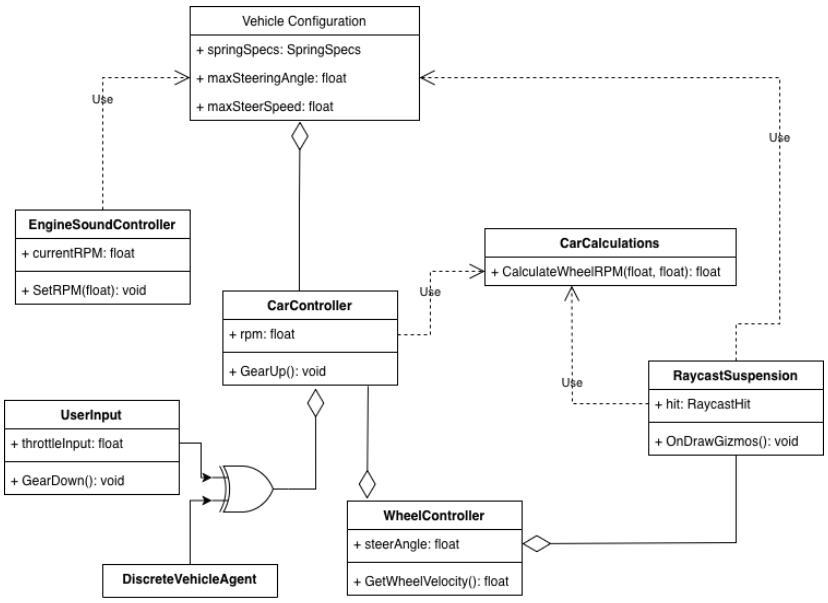


Figure 3: UML Diagram for the main classes related to vehicle physics

Features of the end system

Raycast Suspension

I Implemented a basic suspension system using Unity's Raycast system, by using empties places in the vehicles wheel wells, I could point a raycast down to measure how far down the ground is from the top of the wheel well, and use the position of the empties as "bump stops" where the spring cannot force the top of the wheels to contact the body of the vehicle, and the full force of the wheels is applied to the body as the suspension cannot absorb any more of it due to the spring being completely compressed. A variable isContactingGround is also used to communicate to other vehicle physics subsystems to halt any respective calculations to improve computational efficiency.

The Raycast is used to measure the distance from the wheelwell of the vehicle to the ground where the wheel would contact the ground. I used this distance to calculate the force exerted on the body at each corner of the vehicle. By using Hooke's Law, with a known spring constant and distance, I could calculate the force exerted on the body.

However, using a spring by itself led to extreme vibrations and rocking of the vehicle when any forces are applied to the vehicle through the wheels.

Therefore, I added a damping system, to replicate the function of a shock absorber or damper in a real car's suspension. This damping system is implemented by reducing the force applied by the spring with a known damping rate and the velocity of the spring's

extension / compression. Linking the damping force to the speed of the spring both improves the perceived realism of the system to the end user.

Furthermore, to reduce unnecessary calculations, no suspension calculations are made when the raycast cannot contact another object, as therefore the suspension cannot impart any forces and continuing to run the calculations every physics tick is computationally wasteful.

Tyre Grip Model

In my Implementation of a tyre grip model, I broke up the two frictional forces associated with tyres by their vectors, with one being used for shearing forces and the other for rolling forces. This has the benefit of already simulating the “slip angle” needed to make the vehicle turn, as the lateral turning force is generated by my tyre grip model, and turns can be made by rotating the empties controlling the front suspension, similar to how a real vehicle’s steering system functions.

Maximum tyre grip is calculated to be a function of the normal force applied to the tyre (from the weight of the vehicle) and the co-efficient of friction (different between the two vectors). Normal force is calculated by the Raycast Suspension system and passed into the WheelController class, where the tyre friction calculations are done. The two co-efficients of friction are set variables that do not change and are taken from a separate VehicleConfiguration class.

By performing frictional calculations in local space as opposed to world space, the grip model will function as usual irregardless of the orientation of the associated suspension empties, this allows the turning of the wheels to be done by rotating the empties as opposed to modifying values in the friction model itself, which simplifies steering implementation and makes debugging far simpler as the angle of a given tyre is directly linked to the rotation of the suspension empty.

As it is significantly easier to roll a tire than to slide it against the ground, I used two different frictional co-efficients, one for shearing and one for rolling. This difference in friction generated the lateral turning force required for the tyres to make the vehicle turn.

With each physics tick, the distance travelled by the wheel per tick is measured and this is used to calculate a unit vector to find the direction in which the car is travelling.

As mentioned in the Raycast Suspension Implementation section above, tyre grip calculations are only done when the raycast can reach the ground, this further improves computational efficiency by reducing the number of unnecessary calculations done, as the tyres cannot grip the ground if they are suspended in the air.

Engine and Gearbox Simulation

I used two separate mathematical functions to create a torque curve, where engines usually have a band of efficient operation. I could demonstrate that by having a sharper “upper curve” to have a more dramatic reduction of torque at higher RPMs, where the engine is close to reaching its redline.

To calculate the force applied at the wheels, the RPM was put into the function and a torque value calculated, as engines do not create the same amount of torque at every RPM level, I used quadratic curves to model a rising and falling torque level.

For the transmission, I used an array of gear ratios and a fixed final drive ratio, this is used to calculate the force output at the wheels and allows the vehicle to have harder acceleration at lower gears at the cost of a lower vehicle top speed, and vice versa for higher gears.

For gear changes I opted to use unity’s event system as opposed to having the GearUp() and GearDown() methods called directly by a user script. As these methods do not need to be called consistently nor are they needed for physics calculations, unity’s event system was perfect to handle the inputs related to manually changing gears.

Furthermore, when the gears are changed, the engine needs to adapt to its new rpm relative to wheel speed, so a linear interpolation is done using the wheel’s RPM as a base for where the engine RPM should be. This helps keep the engine RPMs in line with the vehicle's velocity, and allows the engine to rev down slowly when the user reduces throttle and the vehicle starts to decelerate.

To build on the transmission logic, I added two automatic transmission modes, Automatic and Simple Automatic. Both of these modes work by having an upshift target and a downshift target. When the engine rpm reaches the upshift target, the GearUp() method is called automatically. Whereas, when the engine rpm reaches a threshold such that the engine’s rpm in the lower gear at the same vehicle speed is below the downshift threshold, the GearDown() method is called.

The main advantage of using a downshift threshold calculated using the potential rpm of the engine in the lower gear as opposed to a fixed rpm in the current gear, is that the transmission is more aggressive when downshifting, keeping the engine in the rpm range with the highest torque, and removing the “sluggish” feeling of driving a vehicle at low rpms.

The only difference between the Automatic and Simple Automatic modes is that whereas the Automatic transmission mode requires the user to shift down from first gear to reverse when wanting to reverse the car and vice versa, Simple automatic will automatically put the vehicle in reverse when below a certain speed threshold and the brakes are being held.

This gives the user the option between the granular control of having the brake button being only used to slow the car down, or to simplify the control setup in exchange for losing some control over the vehicle.

Lastly, by building a simplified automatic option, I could use this option for all vehicles controlled by my agents, simplifying the action space of the model and reducing training time as a result without impacting performance or the user experience to a noticeable degree.

User Interface / Heads Up Display

For the Heads up display (HUD), I opted to use a single UIHandler class to aggregate all relevant information from the CarController, CheckpointSystem, LapTimer, and VehicleConfiguration classes. This single class controlling all HUD elements removes all the logic from other classes, preventing fractured logic across multiple classes making debugging significantly harder.

For cross-class communication I opted to use polling instead of an Observer Pattern. As the UI handler is required to update the HUD once every frame, I Opted to have the CarController class update the UIHandler every frame with all relevant vehicle information. Whereas, other less time sensitive information such as lap and sector times are updated when the information changes. This balances the computational overhead with polling data with the efficiency of only sending updates for non frame critical data, such as lap times.

Furthermore, to reduce coupling, the CarController system is also able to work without an associated UIHandler class, this is used for ML controlled cars that do not have any HUD.

The global user interface was implemented using a modular screen-space system, where each menu would be assigned its own independent screenspace, which provided the ability to hide and show menus on the fly by simply activating or deactivating the menu's screen space. This also allowed me to implement menus using prefabs, which cuts down on duplicate code and assets down massively, and provides a single source of truth (SSOT) on the format of a given menu type.

Furthermore, the UIController class is also used to handle input mapping switches, by having the ability to switch the current input map from driving to UIControl automatically, and being able to switch the input map back to Driving when a given menu prefab calls for it. This allows pause and play menus to be built as prefabs as all functions are handled by the UIController, which further cuts down on duplicate code.

I utilised unity's event system to reduce coupling between the UI controller class and the prefabs of menus, as each button's function was assigned by an enum, which is then given an event subscription to the event the button should invoke by the sceneBootstrapper class upon scene load.

All Scene switching is also handled by the scenes UIController, where buttons calling These functions are all handled by a MainMenuUIController, with a separate InGameUIController class for in-game menus, as the two systems have separate requirements and separating the classes allowed each class to specialise for its purpose.

Machine Learning Opponents

To Implement my Machine Learning Opponents I utilized the Unity ML-Agents plugin, using a Proximal Policy Optimization (PPO) algorithm as the basis of my agent's training. My main focus in training this agent was to build a model that could understand the complex paths of a track efficiently through a reward based feedback loop.

My agent has many sources to observe, the most important being the raycast perception sensor, used to detect obstacles around the vehicle. By utilising a series of raycasts protruding from the centre of the vehicle, the agent can form a rudimentary "map" of its surroundings.

Additionally, the agent is also fed information about its distance to the next checkpoint and angle to the next checkpoint. This can be used as a "compass" by the agent to locate the next checkpoint.

Furthermore, all observations are normalised to reduce the mathematical impact of large values such as the distances between checkpoints, as the neural network may suffer from Feature Dominance, where the model may ignore smaller changes such as vehicle angle to checkpoint, simply due to the difference in size in the numbers. Therefore, by attempting to scale all observations such that they roughly line up with a [-1, 1] scale, this issue can be sidestepped and the model will not be focussed on overly large data values. Another advantage of this is faster convergence, where smoother, smaller mathematical values improve data consistency and speed up model training.

Finally, I added heuristics to the model, where I can take control of the model and use it to test reward balancing. This allowed me to debug and test to see if a training level was suitable for reinforcement learning before using the level to train my agent. This significantly reduced wasted compute time from agents being taught using stages that were too reward sparse, or dense.

Modular Vehicle System

As mentioned previously, according to the Single Responsibility Principle, I opted to break up all vehicle functions into individual classes that can form one-to-many relationships between each other. This not only improved the maintainability and readability of my code, as finding the logic controlling a given subsystem was made much easier with smaller classes that were named. Additionally, by favouring composition over inheritance when assembling a functional vehicle, I can avoid the 'God Object' pattern, which is a hallmark of poor software design due to its complex nature despite providing no benefit.

Furthermore, this allows for the removal of "modules" that are unnecessary to the function of a specific vehicle. For example, a User Input class is not necessary for the operation of an agent controlled vehicle, and can simply not be added to the prefab at runtime. This not only reduces the amount of unnecessary code being executed on a given game cycle, but also

reduces the chance of accidental cross vehicle communication, where assets meant only for player controlled vehicles may be accidentally added to an agent controlled vehicle and vice versa.

Additionally, by breaking up my code into modules, I could improve scalability by having the relationships between classes be dynamic, instead of hardcoded into a singular god object. This extendability allows me to reuse code to accommodate more features, such as six wheeled vehicles by simply adding more WheelControllers to the array of wheels in my CarController class.

Furthermore, by isolating the subsystems of my project, testing and chasing down bugs is significantly easier as classes are less coupled and as such do not affect each other unintentionally as opposed to a God object class.

Finally, I implemented Interfaces between my classes to allow the vehicle's “body” to remain agnostic of who is operating the vehicle, the type of data being output to the HUD etc. This abstraction further improves extendability for future features, “Futureproofing” my classes for any additional vehicles or features I may wish to add in future.

Vehicle Audio System

Engine sounds are articulated by mixing a multitude of sound files, by using volume and pitch mixing to combine the sound of the engine at given speeds to provide a dynamic engine sound.

Each audioloop is given a class with an associated audiosource, minimum and maximum RPM, and an optimal RPM where the audioloop should be played without any volume or pitch changes. With the minimum and maximum RPM being used to form a range of RPMs in which the audio loop will be played.

An array of audiosources are created with an individual engine audio loop when a vehicle with an EngineSoundController class is spawned into a scene. When these audiosources are created and added to a gameObject, they are kept in a virtualised state where they are played constantly at zero volume, this allows the audio to be played instantly by increasing the volume of the audiosource. This further removes the issue of clicking or other audio stepping with audioSources being enabled or disabled.

Each audio loop has a range of RPMs for which it will be played, and an optimal RPM where the audio will be played at full volume and with no pitch distortion. An audio loop will only be audible to the end user when the engine RPM is not too far away from the audio loops optimal RPM, saving computational resources as only relevant audio loops are played and others are muted.

When the engine RPM does not perfectly line up with any given RPM engine loop, multiple loops will be played simultaneously, with any audioloop with a range encompassing the current engine RPM being played. Depending on where the engine RPM is compared to

where the optimal engine RPM is for a given audioloop, the audioloop will be pitched up or down to give the illusion of the engine spinning faster or slower. As each audio loop is linearly pitched according to how far the engine rpm is from the audio loops optimal RPM, the user will not hear any harsh stepping associated with the engine loops being too far from the optimal RPM.

By having each audio loop have a large range, this avoids the user hearing a perceived “stepping” when each engine audioloop starts or stops playing. Furthermore, by blending multiple audio loops, the limitations of pitch shifting the engine loops can be masked by the sum of all the audio loops playing complementing each other, which mimics having more audio samples than the game actually has. Additionally, by having an engine loop playing near its optimal RPM, any excessive pitch shifting is “drowned out” and imperceptible to the user.

Unity’s advanced spatial audio engine also makes other audio effects easy to implement as they are natively supported by the engine, such as the doppler effect caused by a vehicle passing the user.

Control Singleton

To handle user inputs, I opted to use an InputActionController (IAC) class with a Singleton Design Pattern. The main advantage of using this is I can apply a DontDestroyOnLoad() modifier to the empty gameObject the script is attached to. By using this, I can create a persistent manager that survives scene changes, and can handle all user inputs, as it is all controlled through the IACSingleton.

Furthermore, by having a single source of truth (SSOT) for what CustomInputActions exist, this makes debugging user inputs far easier, as all Inputs are routed through the IAC instead of having multiple Input controllers for each scene, with redundant input bindings. This also makes updating input bindings for new control layouts or features extremely easy, as the main Custom Input Asset is used to handle controls throughout the whole game
.
Additionally, Separating out Input management to a separate class instead of having it be a part of other classes has the advantage of further decoupling unrelated classes, as they are not reliant on each other for user inputs, but rather reliant on the IAC singleton present throughout the game.

Another advantage of using an IAC is being able to abstract away inputs, by having a separate class to translate a variety of inputs into set actions, where I can ensure my classes are ready to respond to a given action, rather than hard-coded inputs which can make adding other controllers such as gamepads or steering wheels time consuming when dealing with multiple classes that handle inputs. This scalability makes extending my game’s hardware base to consoles or mobile phones, or adding new features far easier, and keeps technical debt[34] to a minimum due to the benefits of using an SSOT for inputs.

Finally, by removing input handling responsibilities from all classes, and putting them into the IAC, duplicate input checking logic and polling can be removed, as all the classes have a single class to read input values from.

Cross Data Class

I Implemented a Cross Data Class with a Singleton design pattern to handle data handling across multiple scenes. As everything is destroyed when switching scenes, I opted to add this class as a component on an empty gameObject with a DontDestroyOnLoad() modifier. This has the advantage of being able to persist when changing scenes, so user options such as the current car, or gauge style can be saved when configured.

An advantage to centralising my user preferences, current car etc is that a SSOT can be established, and all scripts dependent on that data can fetch it from this singleton as opposed to having to set the variables in each class in that set. This also helps with reducing the need for storing data in different script components, as the

Secondly, by using a SSOT to handle user preferences, classes that are dependent on that data can be decoupled from other classes that would have that data, as the relationship to get that data is no longer necessary due to the Cross Data Singleton. Furthermore, by utilising a SSOT pattern, I can ensure data integrity by eliminating circular dependencies, as classes wanting to access any cross scene data will do so by communicating to the Cross Data class and not each other. Additionally, encapsulating all relevant cross scene data into one class with managed access to variables can prevent accidental overwriting.

SceneBootstrapper

To reduce duplication in my scenes, I opted to implement a SceneBootstrapper Class, this class is responsible for spawning in vehicles, and setting up other game related functionality such as the number of laps required in each vehicle's checkpointSystem, and other scene setup functions.

This class is made to have the ability to work in conjunction with the Cross Scene Data class. When the scene is first loaded, the SceneBootstrapper class creates empty gameObjects, on which to add a PositionTracker and UIController Script Components, among other components. The configuration of which is also handled by the SceneBootstrapper.

Furthermore, the SceneBootstrapper is also responsible for enabling the start menu UI element, and switching the active input set to UIControls, along with mapping all event listeners to their respective events in a given UI prefab. By decoupling UI logic from scene logic, through dynamically assigning event subscriptions to buttons, this system allows UI prefabs to have complete autonomy from the scene logic,[44] so long as all the buttons are mapped correctly to their given function, as the functionality is added to them at runtime.

Additionally, the SceneBootstrapper also ensures there is an EventListener Present in the scene, as without an EventListener, any inputs by the user will not be able to be recognised by the IAC singleton class.

Furthermore, By moving all prefab spawning and component injection to the SceneBootstrapper class, all dependencies upon scene loading are rolled up into one class that all classes can communicate with.

Future Work

A primary focus throughout the development of this project was the ability to extend and build upon the feature set built within the scope of the project. One example of this is the modularity of the vehicle system, one scope for future work would be to add more vehicles, using the VehicleConfiguration class and new meshes to create a virtually limitless number of vehicles for the end user to enjoy.

Additionally, Vehicle Customisation was a feature to be added to the project, given the modularity of Unity's prefab system, by building prefabs built up of multiple meshes, the meshes can be enabled or disabled to show a given body panel on the vehicle's chassis. However this feature was cut due to time limitations related to modelling these body panels.

Referring back to the initial project scope, the implementation of multiple "brains" for the machine learning model, by modifying the rewards associated with vehicle speed, collisions and other parameters, the machine learning script written for this project could be adapted with enough compute time to retrain the models to change their behaviour while driving.

Self Evaluation

Technical Achievements and Implementations

Physics and Handling: The Raycast Suspension and Tyre Grip model provided an engaging and realistic basis for my CarController class.

Machine Learning Implementation: The Machine Learning implementation was extremely basic and barely functional as is. The ML controlled vehicle would frequently collide into walls and have issues traversing the track.

HUD and UI: The HUD and UI system are functional albeit basic, with the fundamental system being robust and compatible with UI prefabs to cut down on code duplication.

Optimisation: Due to the modular vehicle system and aggressive refactoring on all logic based classes, the code in this project runs efficiently and does not experience any hanging or glitches when running the physics loop in Unity.

Project Management and Workflow

Scope Management: Unfortunately, due to the large scope of my project, some corners were required to be cut to reach the deadline, most of which being applied to the machine learning opponents, the most time and compute intensive subsystem in this project.

Tools and Pipeline: Overall, I believe that choosing Unity as a game engine was the correct choice, due to the availability of third party resources to learn how to use the engine. Additionally, I also believe my usage of Git was more than appropriate for this project, and enhanced my productivity by providing a safety-net for when feature development went wrong.

Problem solving: One major roadblock I encountered during the development of my project was the training of my Machine Learning Agent. While initial trail runs did show successful training, when training a new model on the more complex car controller, sample sensitivity plummeted and noise was massively increased.

Evaluation of Results

Objectives vs Outcomes

In terms of all research objectives, all research was done to a satisfactory degree such that all gathered sources were enough to empower informed decisions on issues such as Machine Learning algorithm selection, suspension and tyre grip modelling, and engine behaviour simulation.

Furthermore, all my technical objectives were achieved, by utilizing the context gathered from the literature review, being able to complete all designs and build proof of concepts for all classes and features outlined in the initial project specification. This proved useful when making initial observations about how these features would work and how to implement them discretely.

In response to testing related objectives, a robust testing system was built, using Unit Testing to test calculations and Play Testing to test gameplay related systems such as the LapTimer and CheckpointSystem classes. Furthermore, any classes in this project were designed to be well documented and easy to maintain and extend for future developers.

System Performance

The two largest contributors to the overall performance of the system were the physics related systems such as the Raycast Suspension and Tyre Grip models, and the Machine Learning (ML) agent controlled vehicles.
All physics related subsystems performed more than adequately, where all physics calculations were able to be completed within a given FixedUpdate() tick, allowing the physics system to work reliably without causing any slow downs or lag.

However, the Machine Learning Agents were not able to perform as expected, due to random hard stops and aversion to driving towards the next checkpoint, likely a result from noise during the training stage that was not obvious until after training was complete.

Limitations

One of the largest limitations of this project is related to the Machine Learning agent, more specifically, the lack of generalisation in its training data. As training environments have to be built in unity, leading to a lot of manual environment creation which is slow and can still result in biased training environments due to the inherent bias in having a singular human build all the environments.

Additionally, due to the move to a more advanced CarController, the Agent suffered from constant training instability where rewards would plummet down to -800 at times, likely due to driving around the circuit backwards and hitting a wall when reaching a corner.
Additionally, the noise provided from the vehicle's power output being related to the gearing and rpm of the engine, the Agent had too many parameters to keep track of and the number of training steps to build an experienced model quickly spiraled out of control.

Overall Judgement

In conclusion, this project represents a successful proof-of-concept for multiple game related subsystems, particularly related to the custom vehicle physics and overall system architecture. However, the failure of the training and implementation of the ML agents plugin into my project, due to the inability to achieve the intended level of performance limited the overall system's effectiveness as a playable video game. This highlights a trade-off done out of necessity due to over development hardships that reduced the amount of compute and development time that could be assigned to the training of the ML agent.

Critical Analysis

Training time and Environment Complexity

The correlation between environment complexity and the required training duration showed a significant development roadblock. While steps were taken to try reduce complexity for the earliest training sections, such as using Curriculum Learning, I believe that the move to the more advanced car controller further added to noise in the training environment.

Consequently, the agent was still stuck in the exploration phase far longer than expected, which further reduces agent performance given the training steps provided.

Training Set Reward Balancing

Another technical issue related to the training of my machine learning agent was the issue of balancing rewards. Where rewards are given to incentivise or disincentivise the agent's actions, an environment with sparse rewards (such as a track with checkpoints) can cause the agents initial exploration phase to be long, as the agent cannot form a set "path" to achieving rewards. This lack of incremental "shaping rewards" makes convergence within the model much harder and as such will require exponentially more training steps to achieve a model with a given level of "experience".

Deadlines and Time Management

Given personal events, and developmental slowdowns, project milestones were far more inconsistent than I expected, which made consistent development much more challenging. As a result of these, Development time was poorly allocated and led to features needing to be scaled back to reach development deadlines. This adherence to delivering "Minimum Viable Product" (MVP), where technical experimentation was balanced against the pragmatic requirement of delivering a stable and playable game, in order to show a "working concept" of all the features outlined in the project scope, albeit simplified down.

What would I have done differently?

I would have ensured that feature deadlines were more concrete, instead of relying on a vague list of project features with no delivery date. Furthermore, I would have attempted to train the ML model on the improved CarController, instead of training on a basic CarController and having to completely change my approach for a more complex CarController.

Appendix

Word Count (main body, excluding references and figures): 14,066 words.

Chapter 5 - Bibliography

[1] “ML-Agents Theory.” *Unity Documentation*, Unity, docs.unity3d.com/Packages/com.unity.ml-agents@4.0/manual/ML-Agents-Overview.html.

[2] Andres Leonardo Bayona. “Comparative Study of SAC and PPO in Multi-Agent Reinforcement Learning Using Unity ML-Agents.” *Repositorio Institucional Séneca*, Universidad de los Andes, 25 July 2023, repositorio.uniandes.edu.co/entities/publication/e5c4cb0d-2df1-46d6-a1d6-990bc854a465.

[3] Mahmoudi, Reza, and Armantas Ostreika. *Reinforcement Learning for Obstacle Avoidance Application in Unity Ml- Agents*.

[4] Revat Meshram, et al. *NPC Behavior in Games Using Unity ML-Agents: A Reinforcement Learning Approach*. 4 Mar. 2025, pp. 1519–1523, ieeexplore.ieee.org/abstract/document/10956320, https://doi.org/10.1109/autocom64127.2025.10956320.

[5] Almón-Manzano, Laura, et al. “Deep Reinforcement Learning in Agents’ Training: Unity ML-Agents.” *Lecture Notes in Computer Science*, 1 Jan. 2022, pp. 391–400, https://doi.org/10.1007/978-3-031-06527-9_39.

[6] Jurison, Jaak. “Software Project Management: The Manager’s View.” *Communications of the Association for Information Systems*, vol. 2, no. 1, 1999, https://doi.org/10.17705/1cais.00217.

[7] Agile Alliance. “What Is Agile Software Development?” *Agile Alliance* |, 29 June 2015, agilealliance.org/agile101/.

[8] Coram, M., and S. Bohner. “The Impact of Agile Methods on Software Project Management.” *12th IEEE International Conference and Workshops on the Engineering of Computer-Based Systems (ECBS’05)*, 2019.

[9] Scrum.org. “What Is Scrum?” *Scrum.org*, 2020, www.scrum.org/resources/what-scrum-module.

[10] Gamma, Enrich, et al. *Design Patterns, Elements of Reusable Object-Oriented Software*. 1994.

[11] Beck, Kent, et al. “Manifesto for Agile Software Development.” *Agile Manifesto*, 2001, agilemanifesto.org/.

[12] Nystrom, Robert. *Game Programming Patterns*. Genever Benning, 3 Nov. 2014.

[13] “Unity Documentation: UnityEvent .” *Unity Documentation*, Unity, docs.unity3d.com/6000.3/Documentation/ScriptReference/Events.UnityEvent.html.

[14] Miecznik, Rafal. “The Importance and Benefits of Unit Testing.” *CodiLime*, 22 June 2023, codilime.com/blog/unit-testing/.

[15] IBM. “Integration Testing.” *Ibm.com*, www.ibm.com/think/topics/integration-testing.

[16] “Edit Mode vs. Play Mode Tests | Test Framework | 1.1.33.” *Unity3d.com*, 2023, docs.unity3d.com/Packages/com.unity.test-framework@1.1/manual/edit-mode-vs-play-mode-tests.html.

[17] Pratheepa, B. (2010). Modeling and simulation of automobile suspension system. *Frontiers in Automobile and Mechanical Engineering -2010*, 2A. doi:https://doi.org/10.1109/fame.2010.5714866.

[18] N. Jazar, Reza. *Vehicle Dynamics: Theory and Application*. 2008.

[19] Gillespie, Thomas D. “Fundamentals of Vehicle Dynamics Revised Edition.” *Google Books*, SAE International, 2021, books.google.co.uk/books?hl=en&lr=&id=LeybEAAAQBAJ&oi=fnd&pg=PP1&dq=Gillespie.

[20] Hall, Nancy. “Newton’s Laws of Motion.” *Glenn Research Center*, NASA, 27 June 2024, www1.grc.nasa.gov/beginners-guide-to-aeronautics/newtons-laws-of-motion/.

[21] “Tire Model in Driving Simulator.” *Code.eng.buffalo.edu*, code.eng.buffalo.edu/dat/sites/tire/tire.html.

[22] Zeidan, Adam. “Friction.” *Encyclopædia Britannica*, 25 Jan. 2019, www.britannica.com/science/friction.

[23] Opong, Rich. “What Is the Traction Circle? – Fully Explained - FLOW RACERS.” *Flow Racers*, 19 Jan. 2023, flowracers.com/blog/traction-circle-fully-explained/?srsltid=AfmBOooF63Z4D4csiY2XPtYsqBnRYVEa5DiJuSP7mXrKEozDTCweyB1b. Accessed 8 Apr. 2026.

[24] - Unity Documentation. (n.d.). *‘Prepare the ground for raycast Wheel Colliders’*. [online]

[25] Unity Technologies. “Unity - Scripting API: Physics.Raycast.” *Unity3d.com*, 2019, docs.unity3d.com/ScriptReference/Physics.Raycast.html.

[26] Dietterich, Tom. “Overfitting and Undercomputing in Machine Learning.” *ACM Computing Surveys*, vol. 27, no. 3, 1 Sept. 1995, pp. 326–327, dl.acm.org/doi/pdf/10.1145/212094.212114, https://doi.org/10.1145/212094.212114.

[27] Ying, Xue. “An Overview of Overfitting and Its Solutions.” *Journal of Physics: Conference Series*, vol. 1168, no. 2, Feb. 2019, p. 022022, iopscience.iop.org/article/10.1088/1742-6596/1168/2/022022/pdf, https://doi.org/10.1088/1742-6596/1168/2/022022.

[28] GitHub. (n.d.). Available at: https://github.com/Unity-Technologies/ml-agents.

[29] Schulman, J., Wolski, F., Dhariwal, P., Radford, A. and Klimov, O. (2017). *Proximal Policy Optimization Algorithms*. [online] Available at: https://arxiv.org/pdf/1707.06347.

[30] Haarnoja, T., Zhou, A., Abbeel, P. and Levine, S. (n.d.). *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor*. [online] Available at: https://arxiv.org/pdf/1801.01290.

[31] Sutton, H. (2014). Peter Morgan Sutton. *BMJ*, [online] 348(mar31 11), pp.g2466–g2466. doi:https://doi.org/10.1136/bmj.g2466.

[32] Akkaya, I., Andrychowicz, M., Chociej, M., Litwin, M., McGrew, B., Petron, A., Paino, A., Plappert, M., Powell, G., Ribas, R., Schneider, J., Tezak, N., Tworek, J., Welinder, P., Weng, L., Yuan, Q., Zaremba, W. and Zhang, L. (2019). *Solving Rubik's Cube with a Robot Hand*. [online] arXiv.org. Available at: https://arxiv.org/abs/1910.07113.

[33] Mashed, B. and Crolla, D. (2014). *Vehicle Powertrain Systems*.

[34] IBM (2025). *Technical debt*. [online] Ibm.com. Available at: https://www.ibm.com/think/topics/technical-debt.

[35] Khan Academy (2016). *What is elastic potential energy? (article)*. [online] Khan Academy. Available at: https://www.khanacademy.org/science/in-in-class11th-physics/in-in-class11th-physics-work-energy-and-power/in-in-class11-spring-potential-energy-and-hookes-law/a/what-is-elastic-potential-energy.

[36] Global.honda. (2026). *Adaptive Damper System | Honda Technology | Honda Global*. [online] Available at: https://global.honda/en/tech/Adaptive_Damper_System_ADS/.

[37] Muir, A. (2019). *How car springs and dampers work*. [online] How a Car Works. Available at: https://www.howacarworks.com/basics/how-car-springs-and-dampers-work.

[38] Kumar, A. and Wagatsuma, H. (2022). A Kamm's Circle-Based Potential Risk Estimation Scheme in the Local Dynamic Map Computation Enhanced by Binary Decision Diagrams. *Sensors*, [online] 22(19), p.7253. doi:https://doi.org/10.3390/s22197253.

[39] Mechanics 2.8. (n.d.). Available at: https://www.mathcentre.ac.uk/resources/uploaded/mc-web-mech2-8-2009.pdf.

[40] www.sciencedirect.com. (n.d.). *Slip Angle α - an overview | ScienceDirect Topics*. [online] Available at: https://www.sciencedirect.com/topics/engineering/slip-angle-alpha.

[41] www.unrealengine.com. (n.d.). *Unreal Engine | The most powerful real-time 3D creation platform*. [online] Available at: https://www.unrealengine.com.

[42] Godot Engine. (n.d.). *Godot Engine - Free and open source 2D and 3D game engine*. [online] Available at: https://godotengine.org.

[43] Unity Learn. (2026). *Encapsulation in object-oriented programming*. [online] Available at: https://learn.unity.com/pathway/junior-programmer/unit/apply-object-oriented-principles/tutorial/encapsulation-in-object-oriented-programming-2 [Accessed 10 Apr. 2026].

[44] Github.io. (2026). *3.3 Code Decoupling & Abstractions*. [online] Available at: https://carpentries-incubator.github.io/python-intermediate-development/33-code-decoupling-abstractions.html [Accessed 10 Apr. 2026].

[45] Gitlab (2025). *What is CI/CD? | GitLab*. [online] about.gitlab.com. Available at: https://about.gitlab.com/topics/ci-cd/.

[46] Technologies, U. (n.d.). *Physics solutions for game development | Unity*. [online] unity.com. Available at: https://unity.com/solutions/programming-physics.

[47] gewarren (2024). *Managed Execution Process - .NET*. [online] learn.microsoft.com. Available at: https://learn.microsoft.com/en-us/dotnet/standard/managed-execution-process.

[48] Unity Docs. (n.d.). *Introduction to IL2CPP*. [online] Available at: https://docs.unity3d.com/6000.3/Documentation/Manual/il2cpp-introduction.html.

[49] Microsoft (2021). *Visual Studio Code*. [online] Visualstudio.com. Available at: https://code.visualstudio.com/docs/other/unity#_set-vs-code-as-unitys-external-editor [Accessed 10 Apr. 2026].

[50] Microsoft (2021b). *Visual Studio Code*. [online] Visualstudio.com. Available at: https://code.visualstudio.com/docs/editing/intellisense.

[51] docs.unity3d.com. (n.d.). *About Unity Test Framework | Test Framework | 1.1.19*. [online] Available at: https://docs.unity3d.com/Packages/com.unity.test-framework@1.1/manual/index.html.

[52] NUnit Organisation (2024). *nunit/nunit*. [online] GitHub. Available at: https://github.com/nunit/nunit.

[53] GeeksforGeeks (2022). *Single Responsibility in SOLID Design Principle*. [online] GeeksforGeeks. Available at: https://www.geeksforgeeks.org/system-design/single-responsibility-in-solid-design-principle/.

[54] Olbrich, S.M., Cruzes, D.S. and Sjøberg, D.I.K. (2010). Are all code smells harmful? A study of God Classes and Brain Classes in the evolution of three open source systems. *International Conference on Software Maintenance*. doi:https://doi.org/10.1109/icsm.2010.5609564.

[55] Redhat.com. (2021). *Implementing single source of truth in an enterprise architecture*. [online] Available at: https://www.redhat.com/en/blog/single-source-truth-architecture.

Chapter 6 - Project Diary

12/10/25 - 19/10/25

- Continued Work on the Vehicle Suspension Models
- Started Work on the Vehicle Controller Methods

6/10/25 - 12/10/25

- Further Experimentation into avenues to simulate vehicle suspension
- Experimenting with Mesh manipulation for the Car Controller
- Started on the Project Plan, taking into account any barriers I experienced while trying to build a basic suspension system
- Project Plan Completed
- Basic Raycast Suspension Completed

29/9/25 - 5/10/25

- Set up Unity Editor
- Experimented in Unity with Suspension Physics
- Experienced Problems in Unity 6 Regarding Lists, Downgraded to Unity 2022.3 LTS for enhanced stability

Temporary Hiatus

24/11/25 - 30/11/25

- Finally Implemented a Basic Raycast Suspension System to this Git Repository
- Added Damping relative to suspension compression / extension speed
- Started Work on a Vehicle Controller, with frictional forces to enhance vehicle stability from random sliding

1/12/25 - 7/12/25

- Enhanced Vehicle Controller Physics with tyre traction that varies based on the normal force exerted on the tyres by the weight of the car (F = uN)

- Tyre Slip Angle Physics are based on the rotation of the suspension empties, making modelling other vehicle features such as 4-wheel steering, AWD, etc far easier
- Meshes for the Wheels and Calipers are transformed using the height data from the Raycast Suspension System
- Wheel and Caliper Meshes also turn with the Steered Wheels

8/12/25 - 14/12/25

- Worked on Interim Report

16/02/26 - 22/03/26

- Researching Unity's ML agents and how to train the required models for my project

23/02/26 - 1/03/26

- Completed more research on Unity's ML Agents and how to implement it into my project
- Continued Debugging odd tyre grip issues, specifically tyre grip moving the vehicle instead of resisting movement of the vehicle

2/03/26 - 8/03/26

- Fixed Tyre Grip Model issues, which made progressing on the project finally possible
- Made changes to the vehicle controller to allow two scripts, one for player inputs and one for ML agent inputs
- Refactored some code to use unity native features instead of less efficient mathematical features
- Implemented a Basic Checkpoint System
- Refactored CarController.cs

9/03/26 - 15/03/26

- Built a Proof Of Concept ML Agent Controlled Vehicle
- Updated Car Controller With Engine Simulation and Gearing
- Worked On Final Report

16/03/26 - 23/03/26

- Updated Engine Simulation and Gearing
- Added HUD for the player
- Added Main Menu and Scene Switching
- Implement the New Unity Input System
- Build a Bootstrapper Script to Initialise Events in Unity Scenes
- Created a Lap Timer

- Added a Code Style Standard (CSharpier)
- Brought all code inline with CSharpier Standards
- Created a Plain C# Service Class to Handle all Car Calculations
- Implemented UTF Testing Framework
- Implemented a Position Tracker for races
- Worked On Final Report

30/03/26 - 05/04/26

- Added Improvements to CarController class
- Added Engine Sounds To Cars
- Worked On Final Report